

Estructura de Carpetas — IntellOps

Introducción

IntellOps es una plataforma de observabilidad predictiva desarrollada para el laboratorio GIDAS (UTN FRLP), en el marco de una Práctica Profesional Supervisada (PPS). El proyecto es realizado por un equipo de estudiantes de Ingeniería en Sistemas de Información, con un tiempo de desarrollo acotado a 200 horas.

Se trata de un proyecto académico: las decisiones de arquitectura y alcance priorizan que el sistema sea viable de construir y mantener por un equipo de estudiantes en ese plazo, sin dejar de sostener buenas prácticas de ingeniería de software (separación de responsabilidades, testabilidad, escalabilidad futura).

El objetivo central de IntellOps es correlacionar dos fuentes de información que, hasta ahora, se monitorean por separado en el laboratorio: la salud de la infraestructura de backend (servidores, bases de datos) y la experiencia real percibida por los investigadores, becarios y estudiantes que usan las herramientas web del GIDAS desde el navegador. A partir de esa correlación, un algoritmo de detección de anomalías (IA) identifica comportamientos inusuales y notifica al administrador del laboratorio vía Telegram o email.

Estructura completa del proyecto con `src/api/` (el backend) organizado en Clean Architecture. El resto de los módulos (ML, notificaciones, dashboard, agente) quedan como servicios separados, tal cual el diagrama los dibuja.

Elemento del C4	Carpeta	estado
Backend Python	src/api/ (Clean Architecture)	ya existe
Algoritmo deteccion (IA)	src/ml/	nueva
Api Telegram/mail	src/api/infrastructure/notification	nueva
Interfaz WEB(react/Next)	src/frontend	nueva
DB	src/api/infrastucture/db/	Nueva
Pipeline CI/CD	.github/workflows/	ya existe
Modulo QA	tests/	ya existe

Informes de seguridad(CIS, LGTM)	governance/ compliance + docs/security	Nueva
----------------------------------	--	-------

Árbol completo

...

```

intellops/
├── src/
│   ├── api/                # Backend Python — Clean Architecture
│   │   ├── main.py         # arma la app, cablea DI, monta routers
│   │   ├── domain/        # capa 1 — reglas de negocio puras, sin frameworks
│   │   │   ├── entities/   # metric.py, anomaly.py, alert.py, prediction.py
│   │   │   ├── value_objects/ # time_range.py
│   │   │   └── repositories/ # interfaces: metric_repository.py, anomaly_repository.py,
│   │   └── alert_repository.py
│   │       ├── services/    # interfaces: notifier.py, anomaly_detector.py
│   │       └── exceptions.py
│   │
│   ├── application/        # capa 2 — casos de uso
│   │   ├── use_cases/      #get_dashboard_summary.py
│   │   └── dto/            # metric_dto.py, alert_dto.py
│   │
│   ├── infrastructure/     # capa 3 — implementaciones concretas
│   │   ├── db/
│   │   │   ├── sqlite_connection.py
│   │   │   ├── models.py
│   │   │   └── repositories/ # sqlite_metric_repository.py, sqlite_anomaly_repository.py, ...
│   │   ├── ml/
│   │   ├── notifications/
│   │   │   ├── telegram_notifier.py    # implementa Notifier
│   │   │   └── email_notifier.py
│   │   ├── config/
│   │   └── settings.py
│   │
│   ├── presentation/      # capa 4 — entrada HTTP
│   │   ├── routers/       # health.py, ingest.py, metrics.py, anomalies.py, alerts.py, da
│   │   ├── schemas/       # metric_schema.py, alert_schema.py (contratos HTTP, no domain)
│   │   ├── dependencies.py # DI: conecta use cases con sus implementaciones concretas
│   │   └── middleware/    # cors.py, logging.py, auth.py

```

```

|
|
|— ml/          # Algoritmo de detección de alertas mediante IA (servicio propio)
|
|— frontend/    # (React/Next)
|   |
|   |— src/
|   |   |
|   |   |— App.jsx
|   |   |— views/
|   |   |— components/
|   |   |— hooks/
|   |   |— services/api.js
|   |— package.json
|
|— agent/       # recolección de datos desde los servidores monitoreados,
(estó hay que volver a revisarlo)
|
|— ml/          # datasets/experimentos/modelos entrenados (ya existe, vacío)
|— tests/       # Módulo QA — espejo de las capas del backend
|   |
|   |— api/
|   |
|   |— ml/
|   |— frontend/
|
|— .github/
|   |— workflows/
|       |— ci.yml          # Pipeline CI/CD
|
|— docs/        # ya existente (brief, ADRs, research)
|
|— governance/
|   |— compliance.md      # ya cubre CIS Benchmark; sumar LGTM
|
|— openspec/
|   |— specs/architecture/ # components.md, containers.md, interfaces.md
|
|— docker-compose.yml
|— Dockerfile
|— Makefile
|— pyproject.toml

```

Conclusión

Cabe aclarar que la estructura de carpetas presentada en este documento constituye un esqueleto inicial, pensado como punto de partida para ordenar el desarrollo del proyecto. Al tratarse de una propuesta elaborada en una etapa temprana, se apoya en una serie de supuestos que todavía no fueron validados en su totalidad por el equipo, y es esperable que algunos módulos, nombres de carpetas o responsabilidades se ajusten a medida que el proyecto avance y surjan nuevas necesidades técnicas o decisiones de diseño. Por lo tanto, esta organización no debe tomarse como una versión final ni definitiva, sino como una base flexible sujeta a revisión y modificación a lo largo del desarrollo.